Problem 2

Feedforward neural networks and training basics

Q1 (a) With intermediate values computed from a forward pass of a neural network, backpropagation computes the gradients of losses by differentiating with chain rule, such that those gradients are minimized in order to generate an optimized outcome of the neural network, as known as "gradient descent".

Q1 (b) Loss and the computed θ_t (the model's parameter) at state t should be cached from a forward pass. θ_t captures a model's parameters. In a purely linear layer, parameters are activated based on $y = wx + b$, where x is the data sample, y is its label, and the rest are parameters, i.e., represented by θ_t. A neural network is trained to find $\theta_t = \{w, b\}$ by maximizing $\hat{y} = wx + b$. Assume the values increase linearly. To achieve this, mathematically the maxima could be reached when $\frac{dy}{dx} = 0$, i.e., when the gradients stop increasing or changing. $\frac{dy}{dx} = w$ is decreasing and so, we know w is the weight / gradient in a linear layer. A forward pass calculates the change of parameter w. By comparing with the true labels in a training set, it also helps estimating how close the new label derived from w is to the true label. In a whole picture, it is also equivalent to $KL(P||Q) = -\sum_{i=1}^{N} p_i \log \frac{p_i}{q_i}$, where P represents the distribution in the true labels set while Q represents the approximated $\hat{y}$ from the activation function. The KL Divergence is the difference between the true dataset and the approximated dataset, and therefore, represented as the "loss". While the backpropagation evaluates an optimal $\hat{y}^*$ from gradient descent, we need the loss and the intermediate $\theta_t = \{w, b\}$ to be cached from the previous process, which is the forward pass, so that the neural network knows where it could converge from. ReLu, abbreviated as "rectified" linear unit, is an alternative to a purely linear unit which avoid vanishing gradient where $\frac{dy}{dx} < 0$, by finding $\max(0, \hat{y})$ where some $\hat{y}$ function might yield a negative value due to non-linearity which is meaningless to keep in the network's learning.

Mini-batch training and SGD as a noisy estimator

Q2 (a) Given empirical risk (or the loss) = $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} l_i(\theta)$; and, $\hat{g}_B(\theta) = \frac{1}{B}\sum_{i=1}^{B} \nabla l_{ib}(\theta)$, as the change of losses, and so, by definition, $\nabla J(\theta) = \hat{g}_B(\theta)$.

$$\Rightarrow \sum_{i=1}^{N} \nabla\, l_i(\theta) = \frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta)$$

An expectation is equivalent to the mean of all samples from a particular random variable.

$$\Rightarrow \sum_{i=1}^{N} \nabla\, l_i(\theta) = \mathrm{E}_B[\nabla l_{ib}(\theta)]$$

Meanwhile, recall that $\nabla J(\theta) = \sum_{i=1}^{N} \nabla\, l_i(\theta) = \hat{g}_B(\theta)$.

$$\Rightarrow \mathrm{E}[\hat{g}_B(\theta)] = \mathrm{E}_B[\nabla l_{ib}(\theta)] = \nabla J(\theta) \blacksquare$$

Q2 (b) $Var[\hat{g}_B(\theta)] = \mathrm{E}[\hat{g}_B(\theta)^2] - \mathrm{E}[\hat{g}_B(\theta)]^2$. By definition, the curvature (second derivative) of a random variable is the change of change of slope, which also represents its correlation or variance. Therefore, we also have: $Var[\hat{g}_B(\theta)] = \nabla^2 J(\theta)$.

Q2 (c) To see how the gradient changes with respect to B, the batch size, as mentioned in (b), we can derive from the curvature, where $Var[\hat{g}_B(\theta)] = \nabla^2 J(\theta)$.

$$\Rightarrow\ \mathrm{E}\left[(\frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta))^2\right] - \nabla J(\theta)^2 = \nabla^2 J(\theta)$$

$$\Rightarrow \frac{(\frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta))^2}{N} - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

Meanwhile, in machine learning terms, the squared mean of losses from a batch over all data samples is equivalent to variance of losses per sample, i.e., $Var(\nabla\, l_i(\theta))$.

$$\Rightarrow Var\big(\nabla\, l_i(\theta)\big) - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

With the given assumption $Var\big(\nabla\, l_i(\theta)\big) = \sum \theta$, we have:

$$\sum \theta - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

From both sides of the equivalence, the curvature of loss, which is the co-variance of $\hat{g}_B(\theta)$, does not have any relationship with B. This could also be visualized in a mathematical manner of $\nabla f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \cdots \\ \frac{\partial f}{\partial \theta_n} \end{bmatrix}$ given any arbitrary function $f(\theta)$, with parameter set θ. The matrix only depends on the number of features in the model θ. Therefore, the gradient noise covariance changes irrespective of the batch size.

Q3 (a) Training loss decreases because the model starts converging and learning within the training data. Validation loss fluctuates because the model could not adapt the trend in the training set to

the validation set. Thus, this is a phenomenon of **overfitting**, where the model learns too well from the training data (similar to "memorizing" samples) but lacks generalizability on unknown data.

Q3 (b) (i) In terms of data, one could observe from the empirical risk $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} l_i(\theta)$, where N represents the number of samples $l_i(\theta)$ represents losses from each sample, under the model θ. The goal of gradient descent, in any machine learning model, is to reduce $\nabla J(\theta)$, i.e., to find θ when $\nabla J(\theta) = 0$. Mathematically, we then have $\sum_{i=1}^{N} \nabla\, l_i(\theta) = 0$. In this expression, the larger N is, the more loss terms are added to result in 0, and in turn, each $\nabla\, l_i(\theta)$ will become smaller and eventually reach 0. Applying in a machine learning setting, we could provide more samples such that the model has more room to converge better.

Q3 (b) (ii) In terms of the model, one could apply regularization techniques like **early stopping**. Overfitting generally means the training process takes unnecessarily long time. Early stopping interferes the training loop once the gradient of losses reaches or very similar to 0. This avoids the model over-generalizing on the training data while allowing it to just make it converged.

Q3 (b) (iii) In terms of optimization, one could apply a suitable optimizer like Adam or AdamW. From $\nabla J(\theta) = \sum_{i=1}^{N} \nabla\, l_i(\theta)$, one proven way in (a) to minimize it is to increase the number of samples. However, provided that N stays constant, another way is to apply another type of loss function $l_i(\theta)$. The current one is stochastic gradient descent, which updates θ parameters according to gradient of losses. Another type is **Adam** (Adaptive Moment Estimation), which is to compute individual adaptive learning rates for different θ parameters by estimating the first and second moments of the gradients. A variant, **AdamW**, applies weight decay on it.